

38. Leer 10 números enteros, almacenarlos en un vector y determinar si la semisuma entre el valor mayor y el valor menor es un número primo.
39. Leer 10 números enteros, almacenarlos en un vector y determinar si la semisuma entre el valor mayor y el valor menor es un número par.
40. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números de los almacenados en dicho vector terminan en 15.
41. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números de los almacenados en dicho vector comienzan con 3.
42. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números con cantidad par de dígitos pares hay almacenados en dicho vector.
43. Leer 10 números enteros, almacenarlos en un vector y determinar en qué posiciones se encuentra el número con mayor cantidad de dígitos primos.
44. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos de los números almacenados en dicho vector pertenecen a los 100 primeros elementos de la serie de Fibonacci.
45. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números de los almacenados en dicho vector comienzan por 34.
46. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números de los almacenados en dicho vector son primos y comienzan por 5.
47. Leer 10 números enteros, almacenarlos en un vector y determinar en qué posiciones se encuentran los números múltiplos de 10. No utilizar el número 10 en ninguna operación.
48. Leer 10 números enteros, almacenarlos en un vector y determinar en qué posición se encuentra el número primo con mayor cantidad de dígitos pares.
49. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números terminan en dígito primo.
50. Leer 10 números enteros, almacenarlos en un vector y determinar cuántos números de los almacenados en dicho vector comienzan en dígito primo.

## Capítulo 10

### Matrices

#### 10.1. Definición

Una matriz es un conjunto de datos organizados en forma de filas y columnas en donde para referenciar cada dato necesitaremos establecer claramente en qué fila y en qué columna se encuentra. Tomemos la siguiente matriz de datos enteros como ejemplo.

Nro. de fila  (1) (2) (3) (4) (5) (6) ← Nro. de columna

|     |    |    |     |    |     |    |
|-----|----|----|-----|----|-----|----|
| (1) | 10 | 32 | -61 | 24 | 19  | 66 |
| (2) | 25 | 36 | 2   | 12 | 100 | 1  |
| (3) | 15 | 20 | 26  | 30 | 35  | 18 |
| (4) | 10 | 21 | 23  | 26 | 85  | 17 |

La anterior es, pues, una matriz de números enteros cuyos datos están organizados en 4 filas y 6 columnas. De esta manera, para ubicar exactamente un dato, solo tendremos que referenciar su posición en cuanto a filas y columnas y sabremos exactamente a qué dato nos estamos refiriendo. Note usted que se necesita tanto de la ubicación exacta de la fila como de la ubicación de la columna para que no exista ninguna duda en cuanto al dato al cual nos estemos refiriendo. Por ejemplo, ¿cuál es el dato que queda en la tercera fila...? Usted seguramente dirá que en la tercera fila están los datos 15, 20, 26, 30, 35 y 18. De manera que no podría preguntarse de manera individual por el dato de la tercera fila, sino por **los datos** de la tercera fila.

De acuerdo a esto, la posición de un dato en una matriz siempre estará determinada por el número de su fila y el número de su columna. Así, por ejemplo, ¿cuál es el dato que se encuentra en la fila 3 columna 2? Pues el número 20.

| Nro. de fila | (1) | (2) | (3) | (4) | (5) | (6) | Nro. de columna |
|--------------|-----|-----|-----|-----|-----|-----|-----------------|
| (1)          | 10  | 32  | -61 | 24  | 19  | 66  |                 |
| (2)          | 25  | 36  | 2   | 12  | 100 | 1   |                 |
| (3)          | 15  | 20  | 26  | 30  | 35  | 18  |                 |
| (4)          | 10  | 21  | 23  | 26  | 85  | 17  |                 |

¿Cuál es el dato que se encuentra en la fila 2 columna 4? Respuesta: el número 12.

| Nro. de fila | (1) | (2) | (3) | (4) | (5) | (6) | Nro. de columna |
|--------------|-----|-----|-----|-----|-----|-----|-----------------|
| (1)          | 10  | 32  | -61 | 24  | 19  | 66  |                 |
| (2)          | 25  | 36  | 2   | 12  | 100 | 1   |                 |
| (3)          | 15  | 20  | 26  | 30  | 35  | 18  |                 |
| (4)          | 10  | 21  | 23  | 26  | 85  | 17  |                 |

¿Cuál es el dato que se encuentra en la fila 1 columna 6? Respuesta: el número 66.

| Nro. de fila | (1) | (2) | (3) | (4) | (5) | (6) | Nro. de columna |
|--------------|-----|-----|-----|-----|-----|-----|-----------------|
| (1)          | 10  | 32  | -61 | 24  | 19  | 66  |                 |
| (2)          | 25  | 36  | 2   | 12  | 100 | 1   |                 |
| (3)          | 15  | 20  | 26  | 30  | 35  | 18  |                 |
| (4)          | 10  | 21  | 23  | 26  | 85  | 17  |                 |

¿Cuál es el dato que se encuentra en la fila 5 columna 3? Respuesta: no existe fila 5. Para cada una de las preguntas, usted puede notar que existe una única respuesta, pues se ha dado exactamente la posición en fila y columna de cada uno de los datos solicitados. Si quisiéramos mostrar en pantalla el contenido de la primera casillita de la primera fila de esta matriz y asumiendo que toda la matriz se llama *M*, entonces simplemente diríamos:

*Escriba M ( 1 , 1 )*

Esto significaría que escriba en pantalla el contenido de la matriz *M* en la fila 1 columna 1. Siempre que se use esta notación, el primer número representará el número de la fila y el segundo representará el número de la columna. Ejecutada esta orden, saldrá en pantalla el número 10. Asimismo, si se da la orden:

*Escriba M ( 3 , 4 )*

estaremos mostrando en pantalla el número que se encuentra en la matriz *M* en la fila 3 columna 4, o sea, que nos referimos al número 30. De esta manera, podremos referenciar todos y cada uno de los datos almacenados en la matriz. Si quisiéramos escribir todos los datos almacenados en la segunda fila, entonces diríamos:

*Escriba M ( 2 , 1 ) , M ( 2 , 2 ) , M ( 2 , 3 ) , M ( 2 , 4 ) , M ( 2 , 5 ) , M ( 2 , 6 )*

Con lo cual saldrían en pantalla los siguientes datos:

25 36 2 12 100 1

Note usted que en la orden

*Escriba M ( 2 , 1 ) , M ( 2 , 2 ) , M ( 2 , 3 ) , M ( 2 , 4 ) , M ( 2 , 5 ) , M ( 2 , 6 )*

el valor que referencia la fila se mantiene constante mientras que el valor que referencia la columna varía desde la primera hasta la última (en este caso, la primera columna es 1 y la última es 6). De tal manera que, si quisiéramos incorporar variables adicionales para simplificar la escritura de los datos de la segunda fila, podríamos decir:

```

.
.
Para Col = 1 hasta 6
    Escriba M ( 2 , Col )
Fin_Para
.
.

```

O escrito con un ciclo *Mientras* sería:

```

.
.
Col = 1
Mientras Col <= 6
    Escriba M ( 2, Col )
    Col = Col + 1
Fin_Mientras
.
.

```

He colocado unos puntos suspensivos para indicar que estos conjuntos de instrucciones son parte de un algoritmo. Note usted que, utilizando apropiadamente un ciclo, su variable índice nos va a servir para referenciar progresivamente las columnas de la matriz y permitir que en pantalla aparezcan todos los números de la segunda fila.

Si quisiéramos mostrar en pantalla todos los números de todas las filas, tendríamos dos formas. La primera e ineficiente que sería:

```

Escriba M ( 1, 1 ), M ( 1, 2 ), M ( 1, 3 ), M ( 1, 4 ), M ( 1, 5 ), M ( 1, 6 )
Escriba M ( 2, 1 ), M ( 2, 2 ), M ( 2, 3 ), M ( 2, 4 ), M ( 2, 5 ), M ( 2, 6 )
Escriba M ( 3, 1 ), M ( 3, 2 ), M ( 3, 3 ), M ( 3, 4 ), M ( 3, 5 ), M ( 3, 6 )
Escriba M ( 4, 1 ), M ( 4, 2 ), M ( 4, 3 ), M ( 4, 4 ), M ( 4, 5 ), M ( 4, 6 )

```

Puede usted notar que, al mostrar los datos por fila, mientras se mantiene constante el valor de referencia de una fila se avanza en las columnas desde 1 hasta 6. Por lo tanto, en una segunda versión ineficiente de este conjunto de instrucciones podríamos escribir lo mismo de la siguiente forma:

```

.
.
Para Col = 1 hasta 6
    Escriba M ( 1, Col )
Fin_Para
Para Col = 1 hasta 6
    Escriba M ( 2, Col )
Fin_Para
Para Col = 1 hasta 6
    Escriba M ( 3, Col )
Fin_Para

```

```

Para Col = 1 hasta 6
    Escriba M ( 4, Col )
Fin_Para
.
.

```

O si quisiéramos escribir los datos, columna a columna, entonces se podría utilizar:

```

.
.
Para Col = 1 hasta 6
    Escriba M ( 1, Col ), M ( 2, Col ), M ( 3, Col ), M ( 4, Col )
Fin_Para
.
.

```

Ya puede usted notar que en cualquiera de los dos casos se repiten instrucciones o conjuntos de instrucciones, lo que hace suponer que con dos ciclos anidados (o sea, adicionando un ciclo interno a la estructura planteada) podremos recorrer todos los datos de la matriz mostrándolos en pantalla.

De acuerdo a esto, podremos recorrer la matriz en mención utilizando los dos ciclos siguientes:

```

.
.
Para Fil = 1 hasta 4
    Para Col = 1 hasta 6
        Escriba M ( Fil, Col )
    Fin_Para
Fin_Para
.
.

```

O también se hubiera podido plantear de la siguiente forma (utilizando ciclos *Mientras* anidados):

```

.
.
Fil = 1
Mientras Fil <= 4
    Col = 1
    Mientras Col <= 6
        Escriba M ( Fil , Col )
        Col = Col + 1
    Fin_Mientras
    Fil = Fil + 1
Fin_Mientras
.
.

```

Deberá el lector desarrollar una prueba de escritorio de cada uno de estos conjuntos de instrucciones con el ánimo de que pueda conceptualizar de una manera más clara el funcionamiento estos ciclos anidados en unión con la idea de una matriz.

En estos ejemplos, hemos asumido que tenemos una matriz de 4 filas y 6 columnas con datos. ¿Qué sucede si en vez de querer mostrar los datos en pantalla quisiéramos leer datos y almacenarlos en la matriz? Entonces lo que tendríamos que hacer es cambiar la orden

*Escriba M ( Fil , Col )*

por *Lea M ( Fil , Col )*

De esta manera cuando, por ejemplo, la variable *Fil* almacene el valor 2 y la variable *Col* almacene el valor 3, la orden *Escriba M ( Fil , Col )* se entenderá como *Lea un entero y guárdelo en la matriz M en la Fila Fil (que vale 2) Columna Col (que vale 3)*, o sea, en la posición ( 2 , 3 ), sabiendo que el primer entero siempre representa la fila y el segundo entero siempre representa la columna.

De esta forma, si utilizamos el siguiente conjunto de instrucciones:

```

.
.
Fil = 1
Mientras Fil <= 4

```

```

Col = 1
Mientras Col <= 6
    Lea M ( Fil , Col )
    Col = Col + 1
Fin_Mientras
Fil = Fil + 1
Fin_Mientras
.
.

```

estaremos leyendo 24 enteros y cada uno de ellos irá quedando almacenado en una "casillita" independiente dentro de la matriz en el orden de las filas, es decir, primero llena todas las "casillas" de una fila, luego continúa con la siguiente fila hasta llegar a la cuarta fila que es la última (considerando que seguimos hablando de la misma matriz 4x6 con la que hemos construido todos los ejemplos hasta el momento).

## 10.2. Características de una matriz

Siempre que vayamos a utilizar una matriz es importante que tengamos en cuenta que los datos en ella están organizados en forma de filas y columnas. Esto es importante porque ello precisamente es lo que justifica la utilización del concepto de matriz dentro de un algoritmo. Una matriz tendrá las siguientes características:

**Nombre.-** Toda matriz, por ser un conjunto de variables, deberá tener un identificador que permita referirse a ella en cualquier momento dentro del algoritmo. Dicho nombre se ajusta a todas las normas y reglas que existen para cualquier variable. Dentro de las particularidades del nombre de una matriz, es importante que este sea altamente mnemónico, o sea, que fácilmente haga referencia a su contenido. Esto será muy útil al utilizar la matriz en un determinado algoritmo.

**Tipo.-** Este representa cuál es el tipo de dato que se va a almacenar en cada una de las casillitas de la matriz. Es importante anotar que tanto una matriz como un vector son conjuntos de datos homogéneos; esto quiere decir que todos los datos almacenados en ellos deben ser del mismo tipo. De tal forma que usted no podrá pensar en que tiene una matriz de 8 filas por 5 columnas y que en las primeras cuatro filas va a almacenar datos de tipo entero y en las otras cuatro va a almacenar datos de tipo real.

**Dimensión.-** Se refiere específicamente a la cantidad de filas y columnas que va a tener la matriz. Es muy útil que tenga en cuenta que "dimensionar" una matriz no es más que determinar claramente cuántas filas y cuántas columnas va a tener antes de comenzar cualquier orden. También es bueno que recuerde que el hecho de que se dimensione una matriz no implica la obligatoriedad de utilizar todas las filas o todas las columnas de ella. Por ejemplo, si usted dimensiona una matriz de 1000 filas por 1000 columnas, puede utilizar dentro de un algoritmo las primeras 15 filas y 12 columnas. Esta es precisamente una de las grandes desventajas de los arreglos, dado que no siempre vamos a saber con certeza cuántas filas y cuántas columnas vamos a necesitar y más de una vez vamos a pecar por exceso, debido a que es posible que dimensionemos más columnas o más filas de las que realmente necesitábamos, o por defecto, pues también nos pueden hacer falta y haber dimensionado menos de las que eran.

**Tamaño.-** Se refiere al resultado de multiplicar la cantidad total de filas por la cantidad total de columnas de una matriz. De esta manera, una matriz de 4 filas por 6 columnas tendrá un tamaño de 24 posiciones. Se denomina tamaño relativo la cantidad total de posiciones que se usan de una matriz. Por ejemplo, si se tiene una matriz 6x9 (recuerde que el primer número representa la cantidad de filas y el segundo, la cantidad de columnas) y de ella usted solo utiliza las 4 primeras filas y las 7 primeras columnas, entonces su tamaño absoluto es 6x9, o sea, 54 posiciones y su tamaño relativo es 4x7, o sea, 28 posiciones. Es obvio pensar que el tamaño relativo es menor o igual que el tamaño absoluto, nunca es mayor.

**Destinación.-** Es muy importante que cuando utilice una matriz dentro de un algoritmo sepa por qué la necesita y tenga sentido incorporar en él un conjunto de campos de memoria con formato de filas y columnas.

### 10.3. Ejemplo con matrices No. 1

#### Enunciado

Leer una matriz 3x4 y determinar en qué posición está el mayor número par.

#### Clarificación del objetivo

Fundamentalmente, se trata de leer 12 datos enteros, almacenarlos en una matriz que tenga 3 filas y 4 columnas y buscar en qué posición está el mayor número par. Recordemos entonces que un número par es aquel que es divisible exactamente entre 2 y siendo N cualquier número  $\text{Si } N / 2 * 2 = N$ , entonces N es un número par asumiendo obviamente que N es además un entero para que al realizar las correspondientes operaciones se aplique aritmética entera y no se generen decimales.

¿Cómo vamos a determinar cuál es el mayor número par? Recorremos una a una las posiciones dentro de la matriz y vamos preguntando en cada una de ellas si su contenido es par, o sea, si se cumple con la decisión  $\text{Si } N / 2 * 2 = N$ , tomando como N cualquier número almacenado en la matriz (o sea, que N es un  $M(i, j)$  para cualquier valor de  $i$  comprendido entre 1 y el tope máximo de las filas y para cualquier valor de  $j$  comprendido entre 1 y el tope máximo de las columnas).

En caso de que algún contenido de la matriz sea par, entonces lo comparamos con una variable que va a almacenar el mayor número par y si dicho contenido es mayor que el que previamente se había almacenado en esa variable, entonces se cambia el dato almacenado en ella por el nuevo valor mayor que se acaba de encontrar. Al tiempo que seleccionamos el mayor par que encontremos, vamos almacenando su posición en dos variables, debido a que tenemos que almacenar tanto la columna como la fila del número que encontremos.

Finalizado esto, mostramos en pantalla lo que nos solicitan, que es la posición del mayor número par y de paso podemos decir cuál era ese número encontrado. La variable que va a almacenar el mayor número par ha de ser inicializada en un número muy pequeño para garantizar la efectividad del algoritmo. Pudiera ser inicializada en cero si tenemos la garantía de que todos los datos leídos van a ser positivos. Para efectos de garantizar la efectividad de este algoritmo, vamos a inicializar dicha variable con el número -30000, un número verdaderamente pequeño.

#### Algoritmo

##### Algoritmo Ejem\_Matriz\_1

##### Variables

```
Entero: M ( 3 , 4 ),           // Almacenará los 12 datos leídos
Fil,                          // Se utilizará como índice para las filas
Col,                          // Se utilizará como índice para las columnas
Mayor_Par,                    // Almacenará el mayor número par hallado
Fil_Mayor_Par,                // Almacenará la fila en donde se encuentra el
                               // mayor número par hallado
Col_Mayor_Par,                // Almacenará la columna en donde se
                               // encuentra el mayor número par hallado
```

##### Inicio

```
Escriba "Digite 12 números enteros" // Solicita los 12 números
Para Fil = 1 hasta 3                // y los almacena en una matriz de 3 filas por
Para Col = 1 hasta 4                // 4 columnas, dejando cada número en cada
Lea M ( Fil , Col ) // una de las posiciones de la matriz
```

```

    Fin_Para
    Fin_Para
    Mayor_Par = -30000      //Se inicializa la variable que va a almacenar
                           //el mayor número par en un número muy muy
                           //pequeño

    Para Fil = 1 hasta 3    //Con estos dos ciclos se recorre la matriz (por
                           //filas) preguntando en cada posición si su
                           //contenido es par y además si es mayor que
                           //el último mayor número par encontrado

    Para Col = 1 hasta 4
    Si M ( Fil , Col ) / 2 * 2 = M ( Fil , Col ) Y M ( Fil , Col ) > Mayor_Par
        Mayor_Par = M ( Fil , Col )      // En caso de que
        Fil_Mayor_Par = Fil              // sea Verdadera se
        Col_Mayor_Par = Col              // almacena tanto el
                                         // número encontrado
                                         // como la fila y la
                                         // columna en donde se
                                         // encontró

    Fin_Si
    Fin_Para
    Fin_Para
    Si Mayor_Par = -30000      //Si esta variable aún almacena -30000 ello
                              //querrá decir que la matriz no tenía números
                              //pares
        Escriba "No existen números pares en la matriz"
    Sino                      // Si esta variable tiene un valor diferente se
                              //mostrará en pantalla no solo el número sino
                              //la posición en donde se encontró
        Escriba "Mayor par ", Mayor_Par, "y está en fila", Fil, "columna", Col
    Fin_Si
Fin

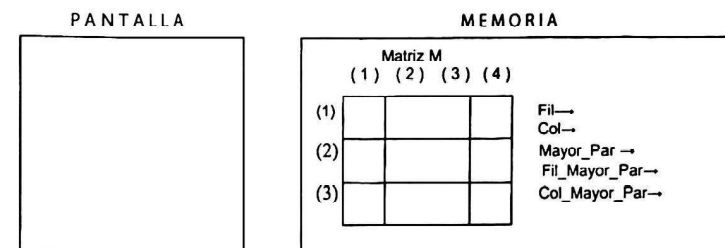
```

**Prueba de escritorio**

Lo primero que haremos será declarar en memoria las variables que van a ser necesarias para el desarrollo de este algoritmo.

**Algoritmo Matrices\_1****Variables**

Entero : M ( 3 , 4 ),  
 Fil,  
 Col,  
 Mayor\_Par,  
 Fil\_Mayor\_Par,  
 Col\_Mayor\_Par,



A continuación, solicitamos los datos del ejercicio y procedemos a leerlos almacenándolos uno a uno en la matriz destinada para ello. Este almacenamiento se hace en el siguiente orden: se inicia un ciclo con la variable *Fil* que tomará valores entre 1 y 3 para hacer referencia a cada una de las filas del vector, dentro de este ciclo se tiene otro ciclo en donde la variable *Col* tomará valores entre 1 y 4 para hacer referencia a cada una de las posiciones que existen en cada fila de la matriz, o sea, como quien dice, para hacer referencia a cada una de las columnas de la matriz. Dentro de estos dos ciclos se ejecutará la orden *Lea M ( Fil , Col )* que significará *Lea un entero y almacénalo en la matriz M en la fila Fil columna Col*.

**Inicio**

Escriba "Digite 12 números enteros"  
 Para Fil = 1 hasta 3  
 Para Col = 1 hasta 4  
 Lea M ( Fil , Col )  
 Fin\_Para  
 Fin\_Para